МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский ядерный университет «МИФИ»
(НИЯУ МИФИ)

ИНСТИТУТ ИНТЕЛЛЕКТУАЛЬНЫХ КИБЕРНЕТИЧЕСКИХ СИСТЕМ

КАФЕДРА «КОМПЬЮТЕРНЫЕ СИСТЕМЫ И ТЕХНОЛОГИИ» (№12)

ОТЧЕТ-СПЕЦИФИКАЦИЯ
КУРСОВОЙ ПРОЕКТ (СХЕМОТЕХНИКА ЦИФРОВЫХ УСТРОЙСТВ)

Семестр 5

Тема: «Конечный автомат протокола и логика управления ONFI». Вариант 3

Студенты Коротаев С. А., Моря К. О. Группа С24-501
ФИО

Руководитель Решетько В. М.
ФИО, должность, звание, степень

Студент _____ Коротаев С. А.
подпись ФИО

Студент _____ Моря К. О.
подпись ФИО

Руководитель _____ Решетько В. М.
подпись ФИО

Оценка _____

Баллы, ECTS _____

«____»_____2026 г.

Москва 2026 г.

ОГЛАВЛЕНИЕ

1. Постановка задачи 3
2. Место модуля в системе 3
3. Буферизация и соответствие команд данным 4
4. Интерфейсы модуля 5
5. Формат пакетов 7
6. Команды, адресация и маскирование 8
7. Внутренняя структура 9
8. Конечный автомат и принцип работы 10
9. Выполнение операций и обработка ошибок 11
10. Тактовая организация и сброс 12
11. Ограничения первой версии 12
12. Согласование с другими вариантами 13
13. Требования к моделированию 14
14. Средства разработки и результат работы 14
15. Приложение. Интерфейсный каркас VHDL 15

1. Постановка задачи

Разработать на VHDL конечный автомат отработки протокола и логику управления контроллера ONFI, вариант 3. Модуль разбирает запросы, организует выполнение операций через ONFI PHY, принимает данные и статусы, формирует ответ и записывает его в буфер.

Основание: задание варианта 3, спецификация Avalon Stream Master варианта 4 и переписка по согласованию вариантов 1, 2 и 4 от 20.10.2026. Документ описывает планируемую реализацию. Результаты моделирования и синтеза в нём не заявляются.

Обязательные функции по заданию:

1. Разбор согласованного с вариантами 1 и 4 пакета: команда, количество данных и служебные поля.
2. Подготовка данных и передача ONFI PHY для записи в NAND Flash.
3. Приём служебных и информационных данных от ONFI PHY.
4. Формирование пакета ответа и запись в буфер.

Модуль не реализует электрические сигналы NAND, временные параметры `WE#/RE#` и внешний Avalon-ST. Физический обмен выполняет PHY, передачу готового ответа по Avalon-ST выполняет вариант 4. Проверка пакета и кодирование ошибок относятся к варианту 3.

2. Место модуля в системе

Вариант 1 Запрос	Буфер запросов Вариант 2	FSM Вариант 3
Вариант 4 Ответ	Буфер ответов Вариант 2	ONFI PHY NAND Flash

Рис. 1. Состав системы. Запрос: 1 → 2 → 3; ответ: 3 → 2 → 4; обмен с PHY двунаправленный

С Максом (вариант 2) согласованы четыре FIFO по 64 бита данных: REQ_CMD, REQ_DATA, RSP_CMD и RSP_DATA. Разрядность и глубина задаются generics. Предыдущее предложение двух общих FIFO заменено этой архитектурой.

Рабочее распределение: request_buffer разделяет входной пакет на служебные слова и тело; response_buffer собирает ответ для варианта 4. Размещение этих функций в варианте 2 предложено Максу, но ещё не подтверждено. Адаптер варианта 3 связывает отдельные очереди с внутренним пакетным интерфейсом автомата. Одновременно обрабатывается один запрос, REQUEST_ID сохраняется.

3. Буферизация и соответствие команд данным

Параметры рабочей конфигурации

FIFO	Данные	Глубина	Содержимое
REQ_CMD	64 бита	16 слов	WORD 0 и WORD 1 запроса
REQ_DATA	64 бита	64 слова	Тело WRITE
RSP_CMD	64 бита	16 слов	WORD 0 и WORD 1 ответа
RSP_DATA	64 бита	64 слова	Данные чтения

Две записи командной FIFO образуют один заголовок. Глубины 16/64/16/64 и G_MAX_LENGTH = 512 байтов выбраны для первой модели; они пока не утверждены другими командами. Глубина считается в словах, длина — в байтах. Для счётчика 0 … ceil(G_MAX_LENGTH/8) выбирается достаточная ширина; при 512 байтах — 7 бит.

Простой безопасный режим: request_buffer допускает один незавершённый запрос. Он сохраняет заголовок в регистрах, тело в REQ_DATA, проверяет транспортные границы и только затем публикует два слова REQ_CMD. Новый пакет блокируется до o_req_release от варианта 3 после записи полного ответа. Это ограничивает параллелизм, но исключает перемешивание команд и данных.

Полный приём не блокирует FIFO при условии ceil(G_MAX_LENGTH / 8) ≤ глубины REQ_DATA. Для выбранных 512 байтов нужно 64 слова. Избыточное тело не записывается сверх лимита: остаток принимается и удаляется до EOP. При увеличении максимальной длины нужно увеличить очередь либо перейти к потоковой обработке; одного изменения LENGTH недостаточно.

Ко второму слову REQ_CMD прикрепляются метаданные: фактически сохранённое число слов тела, empty последнего слова и код ошибки кадра. Они хранятся синхронно с командой в дополнительных битах той же очереди. Это расширяет физическое слово хранения сверх 64 бит данных, не создавая пятую независимую FIFO; такой способ ещё нужно подтвердить с Максом.

response_buffer получает сначала два служебных слова из RSP_CMD, затем ceil(LENGTH / 8) слов из RSP_DATA. Он может выдавать слова по мере появления. Вариант 4 формирует внешние SOP, EOP и empty. Разрешение нового запроса не означает очистку очередей: ранее записанный ответ остаётся в них до передачи.

4. Интерфейсы модуля

Все порты ядра работают в домене i_clk. Таблица 1 описывает внутренний пакетный интерфейс между адаптером варианта 3 и автоматом, а не прямое подключение единственной FIFO. Снаружи адаптер работает с четырьмя очередями.

4.1. Внутренний пакетный интерфейс

Таблица 1. Порты ядра

Сигнал	Напр.	Бит	Назначение
`i_clk`	in	1	Положительный фронт такта
`i_rst_n`	in	1	Сброс, активный 0
`i_req_data`	in	64	Слово запроса
`i_req_valid`	in	1	Слово действительно
`o_req_ready`	out	1	Ядро готово принять слово
`i_req_sop / i_req_eop`	in	1+1	Начало и конец пакета
`i_req_empty`	in	3	Неиспользуемые байты последнего слова
`o_rsp_data`	out	64	Слово сформированного ответа
`o_rsp_valid`	out	1	Ядро предлагает слово ответа
`i_rsp_ready`	in	1	Адаптер может принять слово
`o_busy`	out	1	Запрос обрабатывается или выдаётся ответ
`o_proto_err`	out	1	Ошибка пакета до сброса
Метаданные кадра	in	8+7	i_req_frame_err, i_req_body_words; с WORD 1
`o_req_release`	out	1	Запрос освобождён

Адаптер читает очереди по rd_en при empty = 0 и ждёт rd_valid. Запись выполняется по wr_en при full = 0. Внутри ядра передача происходит при valid = 1 и ready = 1. При valid = 1 и ready = 0 отправитель удерживает слово, valid и признаки. Метаданные действительны с WORD 1; ядро фиксирует их при его приёме. Код кадра: 0 — OK, 2 — BAD_PACKET, 3 — превышение лимита, 6 — TIMEOUT. Счётчики изменяются при передаче. Адаптер разделяет ответ между RSP_CMD и RSP_DATA. o_req_release — импульс освобождения входного запроса после записи последнего слова ответа. Согласованный сброс обязателен для всех очередей.

4.2. Логическая граница с PHY

Таблица 2. Каналы PHY, проектное предложение

Канал	Напр.	Состав и смысл
Команда	FSM → PHY	`cmd_valid/ready`, OPCODE 8, ADDRESS 56, LENGTH 24. Секвенсор варианта 3 преобразует операцию в действия PHY.
Данные записи	FSM → PHY	`w_valid/ready`, data 8, last 1. Передаются только разрешённые байты.
Данные чтения	PHY → FSM	`r_valid/ready`, data 8, last 1. Последний байт отмечает PHY.
Завершение	PHY → FSM	`done_valid/ready`, fail 1, NAND_STATUS 8. Результат подтверждается один раз. При успехе следует после данных; ранняя ошибка прекращает обмен.
Восстановление	FSM ↔ PHY	`recover_req`, `phy_idle`. Запрос удерживается до подтверждения покоя.

4.3. Секвенсор операций

Секвенсор относится к варианту 3 и преобразует операцию контроллера в действия низкоуровневого PHY. Таблица 2 задаёт внутреннюю границу автомата и секвенсора. Конкретные порты между секвенсором и физическим модулем пока не получены от его разработчика.

Проектные этапы выполнения

Операция	Этапы секвенсора
READ	Передать команду и адрес, дождаться готовности, получить заданное число байтов, зафиксировать результат.
WRITE	Передать команду и адрес, выдать проверенное тело, подтвердить программирование, дождаться завершения, прочитать статус.
READ_ID	Передать команду и селектор идентификации, получить нужное число байтов.
RESET / STATUS	Выполнить служебную последовательность и вернуть результат без отдельного тела ответа.

Это схема планируемого управления. Конкретные байты команд, число адресных циклов и интервалы PHY определяются выбранной NAND. Недоступная операция должна возвращать UNSUPPORTED, а не эмулировать успешное выполнение.

5. Формат пакетов

За основу взяты поля из спецификации варианта 4. Порядок байтов: первый байт данных находится в битах 63:56, следующий — в 55:48. VERSION и MSG_TYPE отсутствуют. Направление обмена определяется трактом.

5.1. Заголовок и служебное слово

Таблица 3. WORD 0 запроса и ответа

Биты	Поле	Назначение
63:56	OPCODE	Код операции верхнего уровня
55:40	REQUEST_ID	Идентификатор запроса, копируется в ответ
39:16	LENGTH	Количество байтов операции в запросе; фактическая длина данных в ответе
15:0	RESERVED	Нули

Таблица 4. WORD 1

Пакет	Поле и биты	Назначение
Запрос	ADDRESS 63:8	Адрес операции, 56 бит
Запрос	MASK 7:0	Маска байтов записи, 8 бит
Ответ	STATUS_CODE 63:56	Результат операции, 8 бит
Ответ	NAND_STATUS 55:48	Статус NAND или 0, если неприменим
Ответ	RESERVED 47:0	Нули

Уточнение LENGTH. Для READ поле задаёт число запрашиваемых байтов, но тело запроса отсутствует. Только WRITE содержит LENGTH байтов в запросе. В ответе LENGTH всегда задаёт размер реально передаваемого тела. Это уточнение устраняет неоднозначность исходного описания и требует подтверждения варианта 1.

5.2. Границы и размер пакета

Запрос WRITE: WORD 0, WORD 1, ceil(LENGTH / 8) слов данных.
Остальные запросы: WORD 0 и WORD 1.
Любой ответ: WORD 0, WORD 1, ceil(LENGTH ответа / 8) слов данных.
SOP — на WORD 0; EOP — на последнем слове.
EMPTY = (8 − (LENGTH тела mod 8)) mod 8.

Для пакета без тела EOP приходится на WORD 1 и empty = 0. Например, ответ на READ_ID длиной 5 байтов содержит три слова, на последнем empty = 3. На непоследних словах empty = 0. request_buffer проверяет границы и передаёт результат вместе с фактическим размером тела. Адаптер восстанавливает внутренний пакет. Проверку исходного кадра нельзя заменять синтезированными SOP/EOP.

6. Команды, адресация и маскирование

Коды взяты из спецификации варианта 4 и обозначают операции контроллера. Их физическое исполнение определяется контрактом PHY.

Таблица 5. Планируемые команды

Код	Команда	Запрос	Ответ при успехе
0x01	READ	Адрес, LENGTH > 0; без тела	LENGTH байтов чтения
0x02	WRITE	Адрес, маска, LENGTH > 0 и тело	Статус, LENGTH = 0
0x03	READ_ID	LENGTH > 0; адрес/селектор по PHY	Байты ID
0x04	READ_ PARAMETER	LENGTH > 0; селектор по PHY	Байты параметров
0x05	ERASE_BLOCK	Адрес блока; LENGTH = 0	Статус, LENGTH = 0
0x06	RESET	LENGTH = 0	Статус, LENGTH = 0
0x07	READ_STATUS	LENGTH = 0	Статус в NAND_STATUS, LENGTH = 0

6.1. Адресация и ограничения длины

Для READ/WRITE предлагается байтовый ADDRESS, для ERASE_BLOCK — байтовый адрес начала блока. Геометрия NAND задаёт преобразование в страницу и колонку. Выход за диапазон и пересечение страницы отклоняются до запуска PHY. Разбиение на страницы не поддерживается.

Рабочий G_MAX_LENGTH = 512 байтов ограничивает тело запроса и ответа. G_PAGE_BYTES задаёт размер страницы выбранной NAND; до выбора памяти проверяется на параметризованной модели. Для READ_ID и READ_PARAMETER дополнительно проверяется доступный диапазон конкретной операции. Универсальное значение 65535 из примера варианта 4 не считается размером страницы NAND.

6.2. Маска записи

Предлагаемый базовый режим: MASK = 0xFF, непрерывное тело начинается с ADDRESS, адрес может быть не кратен 8. Последнее слово ограничивается LENGTH. При прочих значениях MASK первая версия возвращает UNSUPPORTED до запуска PHY. Ограничение действует до согласования маски.

Полная поддержка требует согласования битов MASK, действия на слова и пропуска позиций NAND. AND обнуляет байты, но не сохраняет память. Запрещённые позиции требуют адресного пропуска, а не уплотнения потока.

7. Внутренняя структура

Ядро включает декодер, регистры операции, буферы записи и чтения, управляющий автомат, секвенсор операций с адаптером PHY и формирователь ответа. Память буферов ограничена G_MAX_LENGTH. Размещение в блочной памяти определяется синтезом.

Таблица 6. Основные регистры и счётчики

Элемент	**Назначение**
opcode_r, id_r	Команда и идентификатор текущего запроса
address_r, length_r	Адрес и число байтов операции
mask_r	Маска записи
rx_count, phy_count	Принятые байты тела и переданные/полученные байты PHY
rsp_length, tx_count	Размер тела ответа и счётчик выданных байтов
status_r, nand_status_r	Итог операции и статус NAND
state_r, timer_r	Состояние и тайм-аут текущего этапа
write_buffer	Полное тело WRITE до запуска физической записи
read_buffer	Данные чтения до фиксации окончательного результата

WRITE сначала полностью принимается и проверяется. Ошибка структуры пакета, включая i_req_frame_err, не должна запускать программирование NAND. После начала физической записи отмена или откат не гарантируются.

Данные чтения сохраняются до завершения операции. Поэтому заголовок ответа можно сформировать с известными STATUS_CODE и LENGTH. При ошибке чтения частичные данные отбрасываются, LENGTH ответа равен нулю. Это выбранная политика первой версии.

Разрядность счётчиков определяется диапазоном 0 … G_MAX_LENGTH включительно. Вычисление ceil(LENGTH / 8) должно выполняться с расширением разрядности либо через частное и остаток, чтобы LENGTH + 7 не переполнялось.

7.1. Правила управления потоком

При заполнении RSP_CMD или RSP_DATA адаптер останавливает ядро: o_rsp_data и o_rsp_valid сохраняются. Новый запрос до завершения ответа не принимается. Если входной поток временно прерывается, регистры пакета сохраняются. Ожидание внешнего ready не расходует тайм-аут операции NAND; таймер PHY действует только на запущенную операцию.

8. Конечный автомат и принцип работы

Таблица задаёт состояния верхнего уровня. Секвенсор внутри варианта 3 разворачивает операцию в команды, адресные циклы, обмен данными, ожидание готовности и чтение статуса через PHY. Ниже приведён верхний уровень; физические задержки обеспечивает PHY.

Таблица 7. Состояния автомата

Состояние	**Действия и переходы**
WAIT_HDR0	Ожидание SOP и WORD 0. Захват команды, ID и длины. Следом WAIT_HDR1; метаданные ошибки принимаются со вторым словом.
WAIT_HDR1	Приём адреса и маски. Приём метаданных кадра. Далее CHECK_REQ или DRAIN/ ответ по коду ошибки.
CHECK_REQ	Проверка команды, длины и адреса. WRITE с телом: RX_WRITE. Остальные допустимые операции: ISSUE_PHY. Ошибка: DRAIN или RESP_HDR0.
RX_WRITE	Приём полного тела в буфер. Проверка числа байтов, EOP и empty. Корректное завершение: ISSUE_PHY. Ошибка: DRAIN/ответ.
ISSUE_PHY	Удержание команды до cmd_valid AND cmd_ready. После приёма: PHY_WRITE, PHY_READ или WAIT_DONE.
PHY_WRITE	Передача LENGTH байтов из буфера с учётом w_ready. После последнего байта: WAIT_DONE. Ранняя ошибка: RECOVER_PHY.
PHY_READ	Приём байтов с контролем длины и last. После последнего байта: WAIT_DONE. Ранняя ошибка: RECOVER_PHY с сохранением причины.
WAIT_DONE	Ожидание итогового статуса. Успех/ошибка NAND: RESP_HDR0. Тайм-аут: RECOVER_PHY.
RECOVER_PHY	Блокировка новых команд. Отмена/сброс PHY через согласованный адаптер. Только после подтверждения покоя: RESP_HDR0.
DRAIN	Удаление фактически сохранённых слов REQ_DATA по метаданным команды. Затем RESP_HDR0. Следующая команда до освобождения текущего запроса недоступна.
RESP_HDR0	Выдача WORD 0 с известными статусом и длиной. После передачи: RESP_HDR1.
RESP_HDR1	Выдача WORD 1. Без тела: WAIT_HDR0. С телом: RESP_DATA.
RESP_DATA	Выдача буфера чтения. После последнего принятого слова: WAIT_HDR0.

Все переходы, связанные с данными, происходят по фактическому handshake. Сброс из любого состояния возвращает WAIT_HDR0. После записи последнего слова ответа адаптер выдаёт o_req_release, o_busy снимается. Внешняя передача ответа может продолжаться.

9. Выполнение операций и обработка ошибок

9.1. Примеры последовательностей

WRITE 9 байтов. Принимаются два служебных слова и два слова данных, на последнем EOP = 1 и empty = 7. После проверки запускается PHY, передаются девять байтов. После подтверждения завершения выдаются два слова ответа с LENGTH = 0.

READ 16 байтов. Запрос содержит только два служебных слова, EOP стоит на WORD 1. PHY возвращает 16 байтов и статус. При успехе ответ содержит два служебных слова и два слова данных. Вариант 4 ставит EOP на последнем слове данных.

READ_ID 5 байтов. Запрос содержит два слова. После успешного чтения ответ содержит три слова. Пять старших байтов последнего слова полезные, остальные нулевые; вариант 4 вычисляет empty = 3.

9.2. Коды результата

Таблица 8. STATUS_CODE

Код	Имя	Условие
0x00	OK	Успешное завершение
0x01	INVALID_OPCODE	Неизвестный OPCODE
0x02	BAD_PACKET	Неверная структура, границы, reserved или empty
0x03	INVALID_ARGUMENT	Недопустимые адрес или длина
0x04	UNSUPPORTED	Известная операция/маска не поддержана
0x05	NAND_FAIL	PHY сообщил ошибку NAND
0x06	TIMEOUT	Нет прогресса входного пакета или завершения PHY в заданный срок
0x07	INTERNAL_ERROR	Нарушение внутреннего контракта PHY или счётчиков

Для ошибочного запроса с достоверно принятым WORD 0 формируется один ответ с теми же OPCODE и REQUEST_ID, LENGTH = 0. Если WORD 0 недостоверен, request_buffer фиксирует свою ошибку входа без публикации запроса; ответ с вымышленным ID не формируется. Ошибка NAND сама по себе не устанавливает признак ошибки пакета.

request_buffer удаляет остаток повреждённого кадра. При известном WORD 0 публикует два служебных слова, фактическое число слов тела и код ошибки; отсутствующее WORD 1 заменяет нулём. FSM удаляет сохранённые слова REQ_DATA в DRAIN и формирует ответ. После release допускается следующий запрос.

10. Тактовая организация и сброс

Ядро и логический интерфейс PHY работают в одном домене clk_onfi_ctrl. Пересечение с доменом внешнего интерфейса выполняет вариант 2. Численная частота ядра пока не утверждена. Для проверки разных доменов в переписке предложены 50 МГц и 100 МГц; это параметры тестового окружения, а не требования NAND.

На границе ядра i_rst_n снимается синхронно с i_clk. Верхний уровень и FIFO должны согласованно прекращать обмен при сбросе. Состояние, valid, busy, счётчики и o_proto_err очищаются. Буферы признаются недействительными по счётчикам.

Сброс посреди пакета прерывает его без обязательной выдачи ответа. Очереди запросов и ответов должны сбрасываться согласованно, иначе остаток старого пакета может быть принят за новый. Логический сброс FPGA не считается выполнением команды RESET внешней NAND. После тайм-аута PHY автомат не начинает новую операцию до восстановления физического интерфейса. Адаптер должен предоставить запрос восстановления и подтверждение покоя. Без них допустим только общий сброс.

G_RX_TIMEOUT_CYCLES в request_buffer ограничивает приём внешнего кадра; внутри ядра он контролирует контракт адаптера. G_PHY_TIMEOUT_CYCLES ограничивает ожидание PHY. Конкретные пределы определяются после согласования частоты и выбранной памяти. Ожидание свободного буфера ответа не должно ошибочно давать TIMEOUT NAND.

11. Ограничения первой версии

1. Один принятый запрос до release, ответы строго по порядку; MAX_LENGTH = 512 в первой модели.
2. Полная буферизация WRITE до запуска PHY и READ до выдачи ответа.
3. Размер операции ограничен G_MAX_LENGTH и геометрией памяти; переход через страницу не поддерживается.
4. Базовая запись использует MASK = 0xFF. Другие маски требуют доработки и согласования адресного поведения PHY.
5. ECC, bad-block management, wear leveling и многоканальный доступ не входят в данный объём.
6. Проверка на CYC1000 либо аналогичной Cyclone 10 LP предусмотрена заданием; конкретная NAND и параметры PHY требуют согласования.

12. Согласование с другими вариантами

Подтверждённое решение с Максом: четыре FIFO по 64 бита данных, ширины и глубины задаются generics. Остальное ниже — рабочий проект для реализации, который ещё нужно подтвердить участникам.

Таблица 9. Контроль согласования

С кем	**Рабочее решение**	**Что подтвердить**
Макс, вариант 2	Разделение и сборка в request_buffer/ response_buffer. Глубины 16/64/16/64, один запрос до release.	Владение блоками, метаданные кадра, rd_valid, release и согласованный сброс.
Вариант 1	Два служебных слова; MSB-first; READ без тела, WRITE с телом; предел 512 байтов.	Битовые поля, LENGTH, SOP/EOP/empty, адресация и MASK.
Вариант 4	response_buffer собирает 64-битный поток; внешние признаки формирует Master.	Состав ответа, статусы, максимальная длина и FWFT-адаптер при зарегистрированном чтении.
Разработчик PHY	Секвенсор варианта 3, байтовый обмен, результат и восстановление.	Физический интерфейс, NAND, геометрия, режим, тайминги и параметры generics.
Преподаватель	Максимум 512 байтов для первой модели; MASK = 0xFF как начальный этап.	Допустим ли этап с ограниченной маской; итоговая маска и минимальный набор команд.

MASK = 0xFF в первой модели — ограничение демонстрации, а не отмена требования поддержки маски. Формат MASK и сохранение запрещённых позиций должны быть утверждены до окончательной реализации записи.

В задании требуется задавать версию и скоростные параметры ONFI через generics. Эти параметры относятся к PHY/секвенсору и не заменяются отсутствующим полем VERSION пакетного заголовка. Конкретные значения берутся из выбранной NAND и поддерживаемого PHY.

В общую сборку входят разработка, раздельная верификация, синтез и проверка на CYC1000 или аналогичной Cyclone 10 LP. Задание допускает проверку обмена с NAND либо наблюдение протокола логическим анализатором. Эти результаты предстоит получить, они не заявляются выполненными.

13. Требования к моделированию

Самопроверяющийся testbench содержит источник пакетов, модель PHY с управляемыми задержками и приёмник ответов с остановками. Ожидаемые ответы сравниваются по словам и байтам. Модель памяти проверяет адреса и отсутствие физической записи для отклонённых запросов.

Таблица 10. Минимальные проверки

№	Сценарий	Критерий
1	Сброс при простое, приёме, обмене PHY и выдаче	Сняты valid/busy, старые данные не выдаются
2	Команды с допустимыми аргументами, доступные модели PHY	Команда PHY и ответ соответствуют запросу
3	Длины 1, 5, 8, 9, 16 и предельная	Правильные число байтов, порядок и хвост слова
4	Задержки входа, PHY и заполнение выхода	Нет потерь/дубликатов, удержание valid/data
5	Несколько запросов подряд, разные ID	По одному ответу на запрос, ID и порядок сохранены
6	Ошибочные SOP/EOP/empty/reserved, короткое и лишнее тело	BAD_PACKET, нет запуска записи NAND
7	Неизвестный код, превышение длины, адрес вне диапазона, переход страницы	Верный код ошибки до команды PHY
8	MASK = 0xFF и неподдержанная маска	Верная базовая запись или UNSUPPORTED
9	Ошибка NAND, неполные данные PHY, тайм-аут	Нет частичного успешного ответа; выполнено восстановление
10	FIFO с rd_valid, разные такты и согласованный сброс	Корректная стыковка адаптеров, отсутствие повторных чтений

Дополнительно проверяются все остатки длины по модулю 8, стабильность выходов при остановке, нулевые резервные поля и неиспользуемые байты ответа. При ошибке чтения ответ не содержит частичных данных.

14. Средства разработки и результат работы

Планируются VHDL, Intel Quartus Prime и Questa Sim/ModelSim. Базовая плата задания — CYC1000 с Cyclone 10 LP 10CL025YU256C8G либо аналогичная. Конкретная NAND и доступная плата уточняются для общей сборки.

Результат реализации: исходники ядра и адаптеров, testbench и сценарий запуска, результаты моделирования с проверками, отчёт о синтезе и актуальная спецификация. Текущий этап завершается описанием интерфейсов, алгоритма и плана проверки. Следующий этап — утверждение стыковки буферов и PHY, затем реализация и моделирование.

15. Приложение. Интерфейсный каркас VHDL

Каркас показывает внутреннюю пакетную границу ядра после адаптера четырёх FIFO. Логические каналы PHY по таблице 2 связывают автомат с секвенсором варианта 3. Это проектный интерфейс, подлежащий согласованию, а не готовая реализация.

```
library ieee;
use ieee.std_logic_1164.all;

entity onfi_protocol_fsm is
  generic (
    G_MAX_LENGTH          : positive := 512;
    G_PAGE_BYTES          : positive;
    G_BODY_COUNT_BITS     : positive := 7;
    G_RX_TIMEOUT_CYCLES   : positive;
    G_PHY_TIMEOUT_CYCLES  : positive
  );
  port (
    i_clk       : in  std_logic;
    i_rst_n     : in  std_logic;
    -- Request stream, synchronous to i_clk
    i_req_data  : in  std_logic_vector(63 downto 0);
    i_req_valid : in  std_logic;
    o_req_ready : out std_logic;
    i_req_sop   : in  std_logic;
    i_req_eop   : in  std_logic;
    i_req_empty : in  std_logic_vector(2 downto 0);
    -- Response stream to FIFO adapter
    o_rsp_data  : out std_logic_vector(63 downto 0);
    o_rsp_valid : out std_logic;
    i_rsp_ready : in  std_logic;
    -- Local status
    o_busy      : out std_logic;
    o_proto_err : out std_logic;
    -- Request lifetime / framing sideband
    o_req_release : out std_logic;
    i_req_frame_err : in std_logic_vector(7 downto 0);
    i_req_body_words : in
      std_logic_vector(G_BODY_COUNT_BITS-1 downto 0);
    -- Operation channel to local sequencer
    o_cmd_valid : out std_logic;
    i_cmd_ready : in  std_logic;
    o_cmd_op    : out std_logic_vector(7 downto 0);
    o_cmd_addr  : out std_logic_vector(55 downto 0);
    o_cmd_len   : out std_logic_vector(23 downto 0);
    -- Write bytes
    o_w_valid   : out std_logic;
    i_w_ready   : in  std_logic;
    o_w_data    : out std_logic_vector(7 downto 0);
    o_w_last    : out std_logic;
    -- Read bytes
    i_r_valid   : in  std_logic;
    o_r_ready   : out std_logic;
    i_r_data    : in  std_logic_vector(7 downto 0);
    i_r_last    : in  std_logic;
    -- Completion and recovery
    i_done_valid : in  std_logic;
    o_done_ready : out std_logic;
    i_done_fail  : in  std_logic;
    i_nand_status: in  std_logic_vector(7 downto 0);
    o_recover_req: out std_logic;
    i_phy_idle   : in  std_logic
  );
end entity onfi_protocol_fsm;
```